

Jetson Nano Realtime Web Dashboard Architecture

Goal

Create a web-based application where:

- Jetson Nano captures USB webcam video
- Video is streamed in realtime to laptop browser
- Browser UI contains:
 - Live video window
 - GPIO control buttons
 - Device status
- Backend uses:
 - GStreamer
 - OpenCV CUDA
 - FastAPI
 - WebRTC/WebSocket

Recommended Architecture

```
USB Webcam
  |
  v
GStreamer Pipeline
  |
  v
Jetson Hardware Decode
  |
  v
OpenCV CUDA Processing
  |
  v
FastAPI Backend
  |
+---- WebRTC Video Stream
  |
+---- GPIO REST APIs
  |
+---- WebSocket Events
  |
```

Technology Stack

Backend (Jetson Nano)

Component	Technology
API Server	FastAPI
Camera Pipeline	GStreamer
GPU Processing	OpenCV CUDA
GPIO	Jetson.GPIO
Video Streaming	aiortc (WebRTC)
Async Runtime	Uvicorn
Containerization	Docker

Frontend

Component	Technology
UI	React
Styling	TailwindCSS
Realtime	WebSocket
Streaming	WebRTC

Recommended Project Structure

```
jetson-dashboard/  
|  
├─ backend/  
|   └─ app/  
|       └─ main.py  
|       └─ camera.py
```



```
python3-opencv
gststreamer1.0-tools
gststreamer1.0-plugins-base
gststreamer1.0-plugins-good
gststreamer1.0-plugins-bad
gststreamer1.0-plugins-ugly
gststreamer1.0-libav
```

Python Packages

```
pip3 install fastapi uvicorn
pip3 install aiortc
pip3 install opencv-python
pip3 install numpy
pip3 install jetson-gpio
```

Verify Camera

Check webcam:

```
ls /dev/video*
```

Test using GStreamer:

```
gst-launch-1.0 v4l2src device=/dev/video0 ! xvimagesink
```

Optimized GStreamer Pipeline

USB Webcam Pipeline

```
GST_PIPELINE = (
    "v4l2src device=/dev/video0 ! "
    "image/jpeg,width=1280,height=720,framerate=30/1 ! "
    "jpegparse ! "
    "nvjpegdec ! "
    "nvvidconv ! "
```

```
"video/x-raw, format=BGRx ! "  
"videoconvert ! "  
"video/x-raw, format=BGR ! "  
"appsink drop=1"  
)
```

OpenCV CUDA Processing

camera.py

```
import cv2  
  
GST_PIPELINE = (  
    "v4l2src device=/dev/video0 ! "  
    "image/jpeg,width=1280,height=720,framerate=30/1 ! "  
    "jpegparse ! "  
    "nvjpegdec ! "  
    "nvvidconv ! "  
    "video/x-raw, format=BGRx ! "  
    "videoconvert ! "  
    "video/x-raw, format=BGR ! "  
    "appsink drop=1"  
)  
  
cap = cv2.VideoCapture(GST_PIPELINE, cv2.CAP_GSTREAMER)  
  
def get_frame():  
    ret, frame = cap.read()  
  
    if not ret:  
        return None  
  
    gpu_frame = cv2.cuda_GpuMat()  
    gpu_frame.upload(frame)  
  
    resized = cv2.cuda_resize(gpu_frame, (640, 480))  
  
    processed = resized.download()  
  
    return processed
```

GPIO Control

gpio_control.py

```
import Jetson.GPIO as GPIO

GPIO.setmode(GPIO.BOARD)

LED_PIN = 12

GPIO.setup(LED_PIN, GPIO.OUT)

def led_on():
    GPIO.output(LED_PIN, GPIO.HIGH)

def led_off():
    GPIO.output(LED_PIN, GPIO.LOW)
```

FastAPI Backend

main.py

```
from fastapi import FastAPI
from gpio_control import led_on, led_off

app = FastAPI()

@app.get("/")
def root():
    return {"status": "running"}

@app.post("/gpio/on")
def gpio_on():
    led_on()
    return {"gpio": "ON"}

@app.post("/gpio/off")
```

```
def gpio_off():  
    led_off()  
    return {"gpio": "OFF"}
```

WebRTC Streaming

Why WebRTC

Recommended because:

- browser native
- low latency
- scalable
- optimized for realtime

Expected latency:

- 30ms to 120ms

Frontend Architecture

Dashboard Layout

```
+-----+  
|               Jetson Dashboard               |  
+-----+  
|   Camera Stream   |   GPIO Controls   |  
|                   |   [LED ON] [LED OFF]   |  
|                   |                   |  
+-----+  
|               Device Status / Logs               |  
+-----+
```

React Frontend

Install

```
npm create vite@latest frontend
cd frontend
npm install
npm install axios
```

Simple GPIO Button Example

```
async function ledOn() {
  await fetch('http://JETSON_IP:8000/gpio/on', {
    method: 'POST'
  })
}
```

Camera Stream Component

```
<video
  autoPlay
  playsInline
  controls={false}
/>
```

WebSocket Support

Use WebSocket for:

- GPIO status
 - alerts
 - sensor data
 - FPS monitoring
 - system temperature
-

Docker Deployment

Backend Dockerfile

```
FROM nvcr.io/nvidia/l4t-base:r32.7.1

RUN apt update && apt install -y python3-pip

COPY requirements.txt .

RUN pip3 install -r requirements.txt

COPY . /app

WORKDIR /app

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Performance Recommendations

Always Use

- hardware encoders
- NVMM buffers
- CUDA operations
- async APIs
- WebRTC

Avoid

- software encoding
- Flask for realtime streaming
- multiple frame copies
- CPU image processing
- MJPEG for production

Recommended Hardware Encoder

Use:

```
nvv4l2h264enc
```

This significantly reduces CPU usage.

Recommended Final Video Pipeline

```
v4l2src  
→ nvjpegdec  
→ nvvidconv  
→ OpenCV CUDA  
→ nvv4l2h264enc  
→ WebRTC  
→ Browser
```

Future Enhancements

AI Features

You can later add:

- YOLO object detection
 - TensorRT inference
 - face detection
 - QR detection
 - motion detection
 - people counting
-

Security Enhancements

Add:

- JWT authentication
 - HTTPS
 - device pairing
 - API tokens
 - role based access
-

Multi-Camera Support

Possible future support:

- USB cameras
 - CSI cameras
 - RTSP cameras
 - ONVIF cameras
-

Recommended Development Phases

Phase 1

- Webcam detection
- GStreamer pipeline

Phase 2

- CUDA frame processing

Phase 3

- FastAPI backend

Phase 4

- GPIO controls

Phase 5

- WebRTC streaming

Phase 6

- React dashboard

Phase 7

- Docker deployment
-

Recommended First Milestone

Build this first:

- Live camera stream in browser
- One GPIO ON/OFF button
- Single FastAPI backend
- GStreamer webcam pipeline

Once stable:

- move to WebRTC
- add CUDA processing
- add AI inference

Useful Debug Commands

Check CUDA

```
python3 -c "import cv2; print(cv2.cuda.getCudaEnabledDeviceCount())"
```

Check GStreamer

```
gst-inspect-1.0 | grep nv
```

Monitor GPU

```
tegrastats
```

Recommended Production Stack

Layer	Technology
Device	Jetson Nano

Layer	Technology
Camera	USB Webcam
Pipeline	GStreamer
Processing	CUDA/OpenCV
Encoding	nvv4l2h264enc
Streaming	WebRTC
Backend	FastAPI
Frontend	React
GPIO	Jetson.GPIO
Deployment	Docker

Final Recommendation

For best long-term architecture:

- Start directly with GStreamer
- Use CUDA from beginning
- Use WebRTC instead of MJPEG
- Use FastAPI instead of Flask
- Keep frontend separate from backend
- Use Docker for deployment

This architecture is scalable enough for:

- robotics
- AI edge systems
- industrial automation
- surveillance systems
- smart factories
- remote monitoring
- autonomous systems